

[SVT Lab] Final Report: Testing Laboratory Concepts

Reza Ahmadi (VR510067)

March 29, 2026

Contents

1	Introduction	3
1.1	Verification vs Testing	3
1.2	The cost of Faults	3
2	Hardware Description Languages: Verilog-AMS	3
2.1	Mixed-Signal Modeling	3
2.2	Key Concepts	4
3	Fault Modeling in Integrated Circuits	5
3.1	Digital Fault Models	5
3.2	Analog Fault Models	5
4	Automatic Test Pattern Generation (ATPG)	6
4.1	Test Classification	6
4.2	Coverage Metrics	6
5	Design for Testability (DFT)	7
5.1	Scan Design	7
5.2	Built-In Self-Test (BIST)	8
5.3	Comparison between ATPG and BIST	8

1 Introduction

The main purpose of this report is to provide a quick review of all the important topics in the testing part of the Systems Verification and Testing course.

1.1 Verification vs Testing

Verification: the main approaches are defined to check the design and the designer's intention, and to assess how a design respects that intention. However, in Testing, we check every device that is made; for instance, we check whether the circuit's output respects our input. In short, we verify the final product only once after all features are implemented, but we test every circuit or device made from the design. Another difference between Verification and Testing is how they are applied: for Verification, we use simulators to mimic design behavior, whereas in Testing, we test every real circuit or device.

1.2 The cost of Faults

One of the most important and notable purposes of testing is to reduce the cost of faults. In fact, if we consider an IC, finding a fault at a lower level (wafer level) is simpler and cheaper than when it is used in a smartphone or a car.

2 Hardware Description Languages: Verilog-AMS

Verilog-AMS (Analog and Mixed-Signal) is an HDL (Hardware Description Language) extension that helps us to combine event-driven simulation (digital) with continuous-time simulation (analog) into a single standard. Verilog-AMS combines two languages (Verilog and Verilog-A) and provides an extension to allow mixed-signal components.

2.1 Mixed-Signal Modeling

Verilog-AMS (Analog and Mixed-Signal) is important because it allows us to simulate complex behavioral systems that integrate digital control logic with analog hardware within a single language. We use mixed modeling for purposes such as:

- **System Integration:** Modern designs, like smart hybrid power car systems or automatic braking, need a digital controller to control analog components like motors, fuel tanks, or sensors.
- **Domain Convergence:** Mixed modeling helps us to connect the digital world with the analog world.
- **Language combination:** Verilog-AMS combines Verilog-HDL and Verilog-A.
- **Simulation Balance:** Mixed Modeling provides a balance between simulation performance (considered by speed) and accuracy, and we can control it between high-accuracy SPICE and high-performance for digital models.

Mixed models are demanded for component modeling, especially when we have hardware with several levels of implementation. They also help us accelerate the simulation process. Furthermore, they support a top-down design process that helps engineers move from abstract architectural models to detailed implementations.

2.2 Key Concepts

- **Mixed-Signal Nature:** It is about Verilog-AMS, which is a combination of Verilog-HDL and Verilog-A. In fact, Verilog-AMS is a bridge between the digital and analog worlds within a single module.
- **Disciplines and Natures:** Verilog-AMS supports disciplines such as thermal, mechanical, and rotational, enabling the simulation of physical processes beyond electrical and electronic ones. Each discipline is related to a nature that defines certain physical characteristics, such as potential (for instance, voltage) and flow (for current).
- **Conservative Behavioral Modeling:** Verilog-AMS uses branch contributions (represented by the `+` operator) to define relationships between nodes. For example, a resistor is modeled by relating voltage and current: $V(p, n) < +r * I(p, n)$. This allows the analog solver to apply Kirchhoff's laws to solve for unknown voltages and currents.
- **Analog Events and Synchronization:** Verilog-AMS introduces special sensitivity triggers, such as `@cross` and `timer`, to synchronize digital and analog domains
- **Accuracy vs. Performance Balance:** It is related to having a balance or trade-off between simulation performance and accuracy.

3 Fault Modeling in Integrated Circuits

Fault modeling is a concept we use to inject intended faults into a circuit or system, such as a digital signal being "stuck-at" a specific value, like 0 or 1, within a chip design. Thanks to this technique, we can test if the circuit's safety mechanisms will successfully detect and handle real-world failures before the physical chip is ever manufactured.

3.1 Digital Fault Models

In digital systems that process discrete data, digital fault models focus on logic errors. The most common type of fault is the Stuck-At fault, in which a specific node in the circuit is permanently stuck at a value of '0' or '1'. Under standards like ISO 26262, these kinds of errors are categorized by interacting with the chip's safety mechanisms, such as:

- **Single Point Faults:** A fault that directly breaks a system's safety goal because it is not protected by any safety mechanism.
- **Residual Faults:** A fault that breaks a safety goal, while it is technically covered by a safety mechanism, but the mechanism fails to detect it.
- **Multi-Point Faults:** Faults that only violate a safety mechanism when another fault has occurred, and it is combined with that.
- **Latent Multi-Point Faults:** A fault occurring inside the safety mechanism and that goes completely undetected by any other safety mechanisms.
- **Detected Multi-Point Faults:** A fault inside a safety mechanism that is successfully detected by a secondary safety monitor.

3.2 Analog Fault Models

Unlike digital circuits, analog circuits deal with continuous signals. A fault in analog circuits is not just a simple 0 or 1; it can be a kind of shift in a waveform. Analog fault models are categorized into two categories:

- **Catastrophic (Hard) Faults:** These faults represent a huge physical damage in the circuit, modeled as either a complete short circuit or an open circuit.
- **Parametric (Soft) Faults:** These faults represent problems in productions. The circuit doesn't completely fail, but a part or component drifts outside its acceptable tolerance range.

4 Automatic Test Pattern Generation (ATPG)

ATPG is an EDA method for finding a specific input or test sequence. When we use it in a digital circuit, it helps us use automatic test equipment to distinguish between correct and faulty circuit behavior due to defects. On the other hand, it provides patterns for testing devices after they are manufactured.

4.1 Test Classification

During the test generation process, ATPG classifies faults into several different categories for evaluating the system, such as:

- **Full (FU):** All of the injected faults inside the design.
- **Detected (DT):** Faults successfully detected by the tool, which will be divided into detected by simulation (DS) and detected by implication (DI).
- **Aborted:** Failures where the tool interrupted the generation of tests because it reached the backtrack limit, leaving them undetected.
- **ATPG Untestable (AU):** Failures for which the test generator cannot find test patterns, but is also unable to prove their redundancy.
- **Redundant (RE):** Redundant failures that the test generator considers impossible to detect.
- **Undetected (UD):** Failures the test generator could not detect and failed to prove untestable, which includes Uncontrolled (UC) and Unobserved (UO) faults.

4.2 Coverage Metrics

For evaluating the ATPG process, we can measure it by using three primary metrics:

- **Test Coverage:** It refers to the percentage of faults detected in the middle of all testable faults.
- **Fault Coverage:** It refers to the percentage of faults detected in the middle of all injected faults.
- **ATPG Effectiveness:** It refers to the ability of the test generator to either successfully create a test for a fault or to definitively prove its non-testability.

5 Design for Testability (DFT)

DFT is a methodology where testing considerations are incorporated during the initial hardware design phase, instead of attempting to develop tests after the design is already frozen.

5.1 Scan Design

Scan design is a Design for Testability (DFT) technique; the main mission of this technique is transforming the sequential logic of a circuit into a shift register, also called a scan chain. the basic idea is:

- Transform FFs in sequential logic into a shift register
- All FFs controllable & observable via serial access
- Many different types of scan FF (too many!)
- Test problem simplified to testing combinational logic
- The best overall DFT approach ever developed

Scan Design: Advantages

- Fully automated DFT process
- Well-supported by CAD vendors
- Sequential circuits become combinational in test mode
- ATPG CAD tools for generating test vectors
- Typically no fault simulation required
- ATPG can identify redundant logic
- Helps minimize design
- Reduced time-to-market
- High fault coverage
 - Near 100% for gate level stuck-at and bridging faults
- Can be applied hierarchically
 - chips $\Rightarrow$ boards $\Rightarrow$ system
- Allows simplified & accurate fault/defect diagnosis & FMA
- Highly structured & provides good basis for BIST

Scan Design: Disadvantages

- Overhead:
 - Pins: 3 (Scan Data In, Scan Data Out, Scan Mode)
 - Scan Data In can be shared with the primary input
 - Scan Data Out can be shared with the primary output
 - Area Overhead: 1 MUX per flip-flop
 - Typically 2-10% total area overhead penalty
 - Performance degradation: added delay through MUX
 - Routing overhead: scan chain connections, Scan Mode
- Long test application time:
 - $\# \text{clock cycles} = \# \text{vectors} \times (\# \text{FFs} + 1) + \# \text{FFs}$
 - Lots of vectors & output responses to store
- Logic must be synchronous
- Difficult to test at system clock speed

5.2 Built-In Self-Test (BIST)

It is a methodology in which dedicated test circuitry is incorporated directly into a chip or board. It automatically generates pseudorandom test patterns to stimulate the logic.

5.3 Comparison between ATPG and BIST

- Test quality
 - **Logic BIST:** High quality; additional detection of un-modeled defects
 - **ATPG:** High quality; additional detection with patterns for small delay, cell-internal, and other defects
 - **Mix:** Highest quality
- Design impact
 - **Logic BIST:** High
 - **ATPG:** Low
 - **Mix:** High
- Test environment
 - **Logic BIST:** Minimal interface; retest in system and burn-in
 - **ATPG:** Needs a tester; manufacturing test
 - **Mix:** All environments
- Design type

- **Logic BIST:** Designs that need retest in system
- **ATPG:** Any digital circuit chip
- **Mix:** Mission-critical, such as automotive and medical
- **Test application time**
 - **Logic BIST:** Short if design is not pseudorandom resistant
 - **ATPG:** Depends on the number of pattern sets (stuck-at, transition, small delay, path-delay, cell-internal) desired
 - **Mix:** Shortest; a combination of logic BIST for quick detection and ATPG to top up the undetected faults